Decision Trees

Decision Trees

- Goal: Classification
- Main Idea: Modeled after flowcharts
  - Ask Yes/No questions until you learn enough to classify the example
  - Similar to 20 questions
  - Is it bigger than a breadbox?
- Strategy: What questions should you ask?

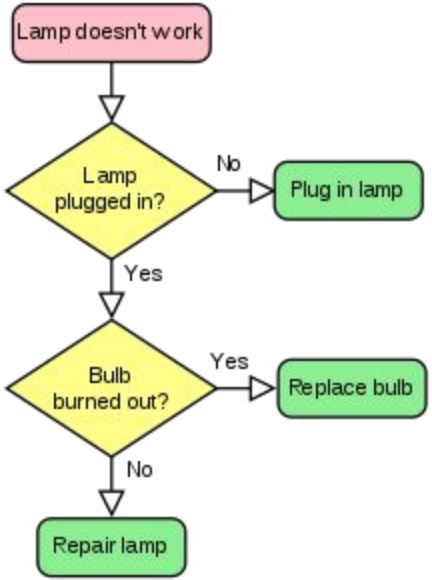


Image Source

Playing Tennis

- Is it raining out?
  - Probably shouldn't play
- Is it really hot?
  - Yes: Maybe, is it also windy?
    - If yes, sure!
    - Otherwise, I'll pass
  - No: Sounds like a nice day, let's play!

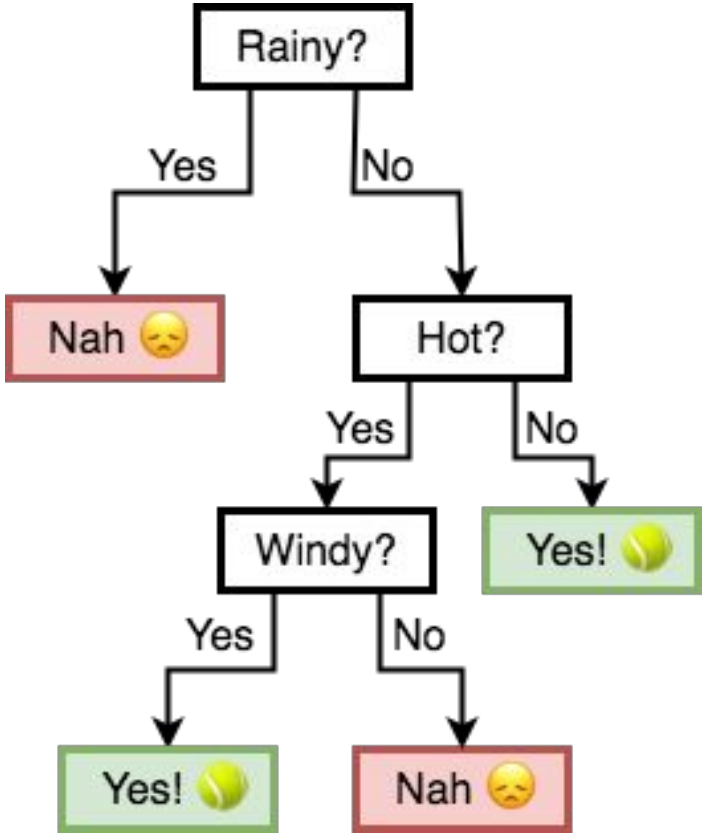
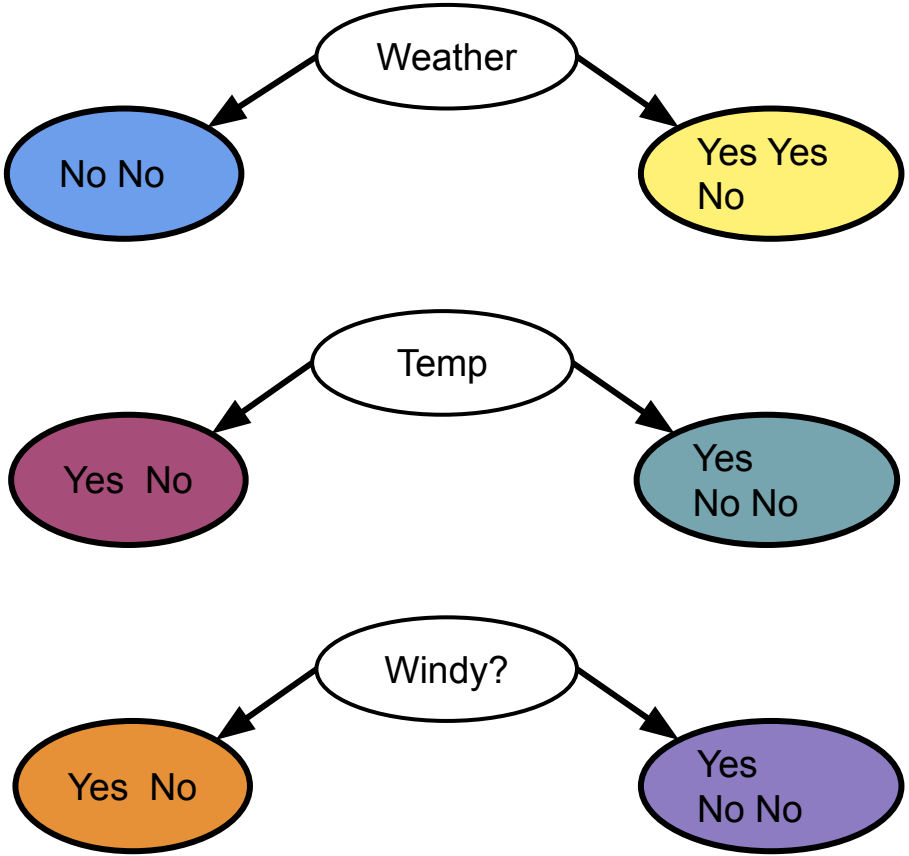
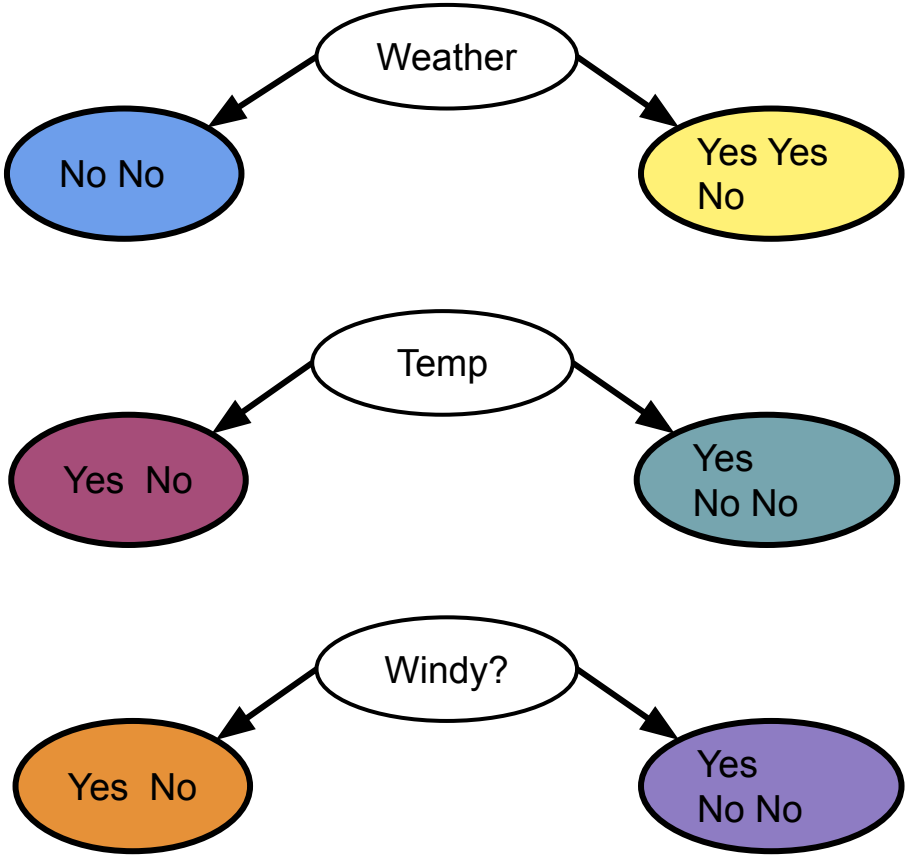


Image Credit: Will Povell (TA '17)

Weather	Temp	Windy?	Play?
Sunny	Hot	Yes	Yes
Sunny	Cold	No	Yes
Sunny	Hot	No	No
Rainy	Cold	No	No
Rainy	Cold	Yes	No

Weather	Temp	Windy?	Play?
Sunny	Hot	Yes	Yes
Sunny	Cold	No	Yes
Sunny	Hot	No	No
Rainy	Cold	No	No
Rainy	Cold	Yes	No





Weather is the best predictor!

Rainy	0% Play
Sunny	66% Play

Hot	50% Play
Cold	33% Play

Windy	50% Play
Not Windy	33% Play

Weather	Temp	Windy?	Play?
Sunny	Hot	Yes	Yes
Sunny	Cold	No	Yes
Sunny	Hot	No	No
Rainy	Cold	No	No
Rainy	Cold	Yes	No

Weather is the best predictor!

Rainy	0% Play
Sunny	66% Play

Hot	50% Play
Cold	33% Play

Windy	50% Play
Not Windy	33% Play

Majority Vote & Misclassification Error

Rainy	0% Play
Sunny	66% Play



don’t play :(



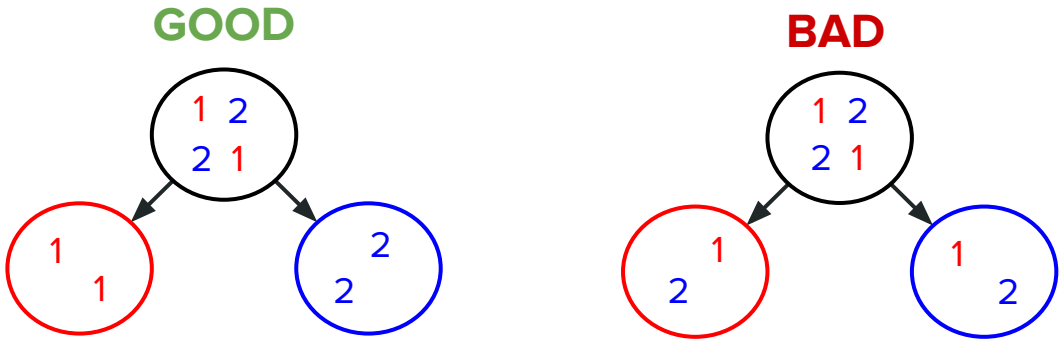
PLAY!

With majority vote

- $\max(p_1, p_2)$ are classified correctly
- $1 - \max(p_1, p_2)$ are classified incorrectly

Here, p_1 and p_2 are the percentages in class 1 and 2 (rainy, sunny), respectively.

Good questions minimize error!



With majority vote, max(p_1 , p_2) are classified correctly => Error = 1 - max(p_1 , p_2)

- p_1 = % belonging to class 1
- p_2 = % belonging to class 2

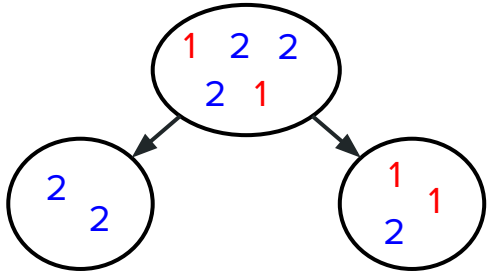
- **GOOD**: Error = 1 - max(100%, 0%) = 1 - 1 = 0
- **BAD**: Error = 1 - max(50%, 50%) = 1 - 0.5 = 0.5

Good questions minimize error!

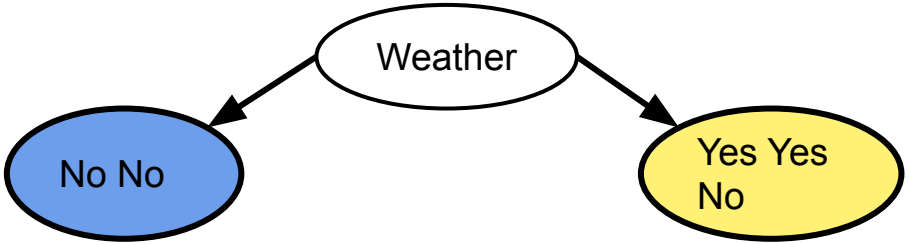
Error(Left) = 1 - max(66%, 33%) = 0.33
Error(Right) = 1 - max(0%, 100%) = 0.0

Weight(Left) = 40%
Weight(Right) = 60%

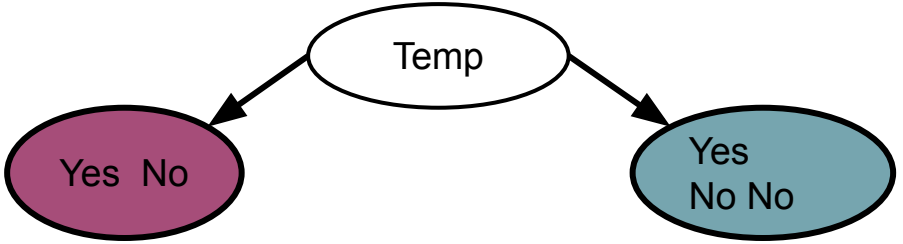
Weighted error = (0.0 x 0.4) + (0.33 x 0.6) = 0.2



Error(Left) = 1 - max(100%, 0%) = 0
Error(Right) = 1 - max(33%, 66%) = 0.33
Weight(Left) = 0.4, Weight(Right) = 0.6
I = (0.4 * 0) + (0.6 * 0.33) = 0.2 Least Error!



Error(Left) = 1 - max(50%, 50%) = 0.5
Error(Right) = 1 - max(33%, 66%) = 0.33
Weight(Left) = 0.4, Weight(Right) = 0.6
I = (0.4 * 0.5) + (0.6 * 0.33) = 0.4

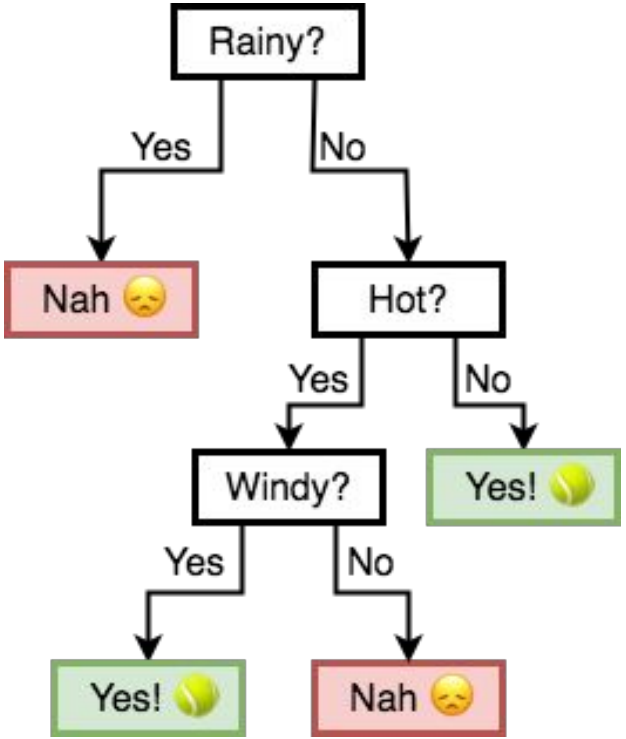


Error(Left) = 1 - max(50%, 50%) = 0.5
Error(Right) = 1 - max(33%, 66%) = 0.33
Weight(Left) = 0.4, Weight(Right) = 0.6
I = (0.4 * 0.5) + (0.6 * 0.33) = 0.4



Greedy Algorithm for Building a Decision Tree

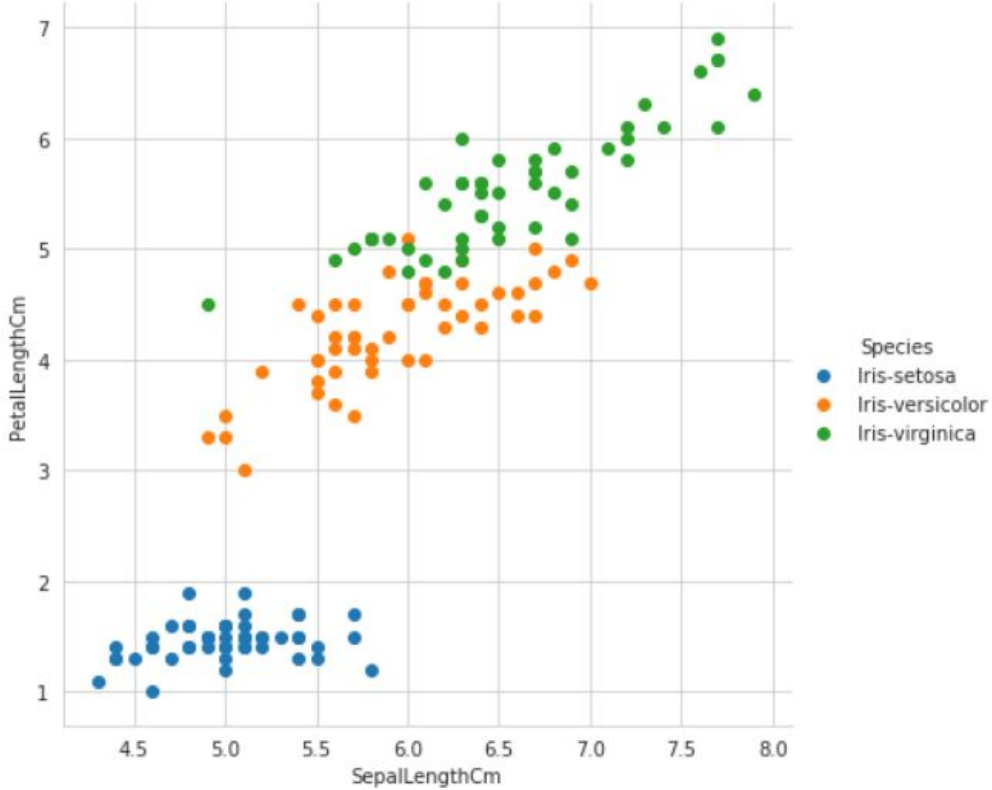
- Start at the root of the tree,
  with all observations
- Score the quality of all questions
  using current set of observations
- Split current set of observations
  based on the question with the best score
- Repeat until all sets of observations
  are contained in just one class
- Classify a new observation by walking the
  tree according to the observation's features

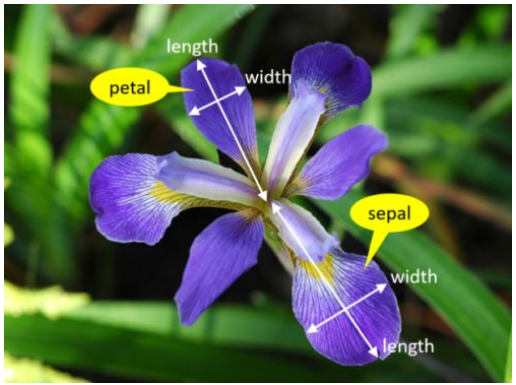


`iris` Dataset

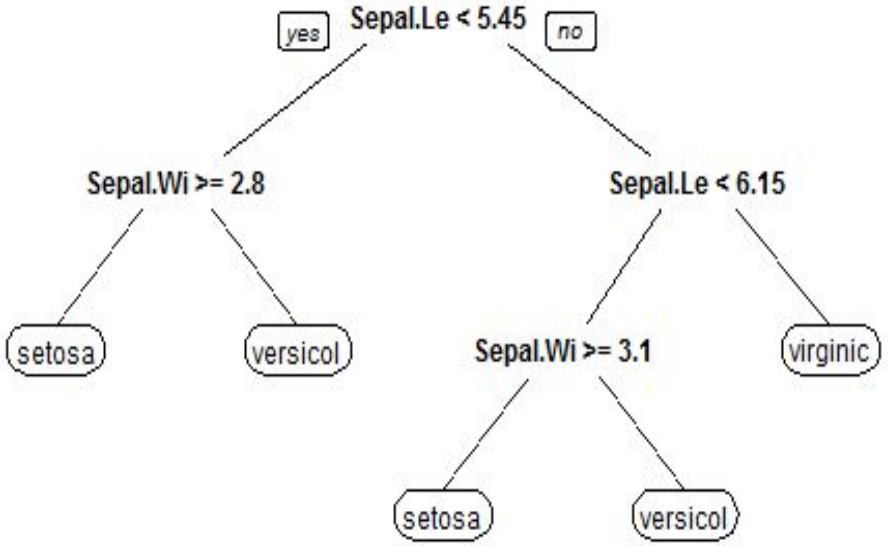
Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
1	5.1	3.5	1.4	0.2	Iris-setosa
2	4.9	3	1.4	0.2	Iris-setosa
3	4.7	3.2	1.3	0.2	Iris-setosa
4	4.6	3.1	1.5	0.2	Iris-setosa
5	5	3.6	1.4	0.2	Iris-setosa
6	5.4	3.9	1.7	0.4	Iris-setosa
7	4.6	3.4	1.4	0.3	Iris-setosa
8	5	3.4	1.5	0.2	Iris-setosa
9	4.4	2.9	1.4	0.2	Iris-setosa
10	4.9	3.1	1.5	0.1	Iris-setosa
11	5.4	3.7	1.5	0.2	Iris-setosa
12	4.8	3.4	1.6	0.2	Iris-setosa
13	4.8	3	1.4	0.1	Iris-setosa
14	4.3	3	1.1	0.1	Iris-setosa
15	5.8	4	1.2	0.2	Iris-setosa
16	5.7	4.4	1.5	0.4	Iris-setosa
17	5.4	3.9	1.3	0.4	Iris-setosa
18	5.1	3.5	1.4	0.3	Iris-setosa
19	5.7	3.8	1.7	0.3	Iris-setosa

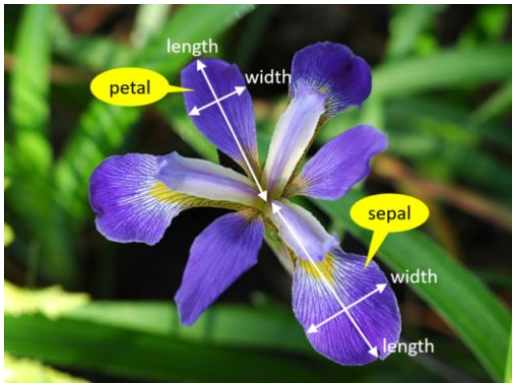
`iris` Dataset, Classified





A decision tree for `iris`





A decision tree for `iris`



Pros of Decision Trees

Pros:

- Interpretable
- Suitable for both quantitative and categorical features
- Suitable for both quantitative and categorical labels (i.e., regression)

Cons of Decision Trees

Cons:

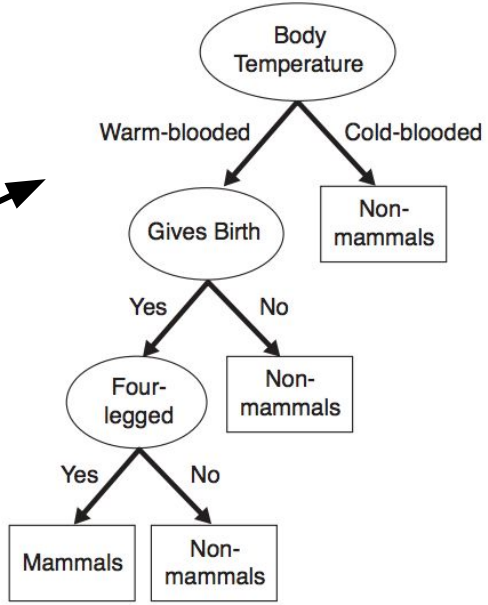
- Easy to overfit!
- Highly variable: a small change in training data leads to a large change in tree

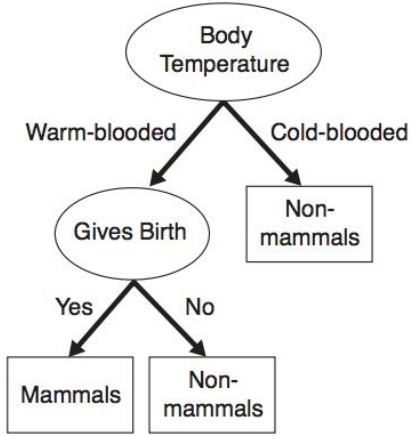
Occam's Razor: The simplest explanation is best!

Einstein: "Everything should be made as simple as possible, but no simpler."

Overfitting

Fits training data perfectly; accuracy is poor on test data!





Accuracy is less than perfect on training data; but tree is simpler, and accuracy is better than the more complex tree on test data!

Image Source

Evidence of Overfitting

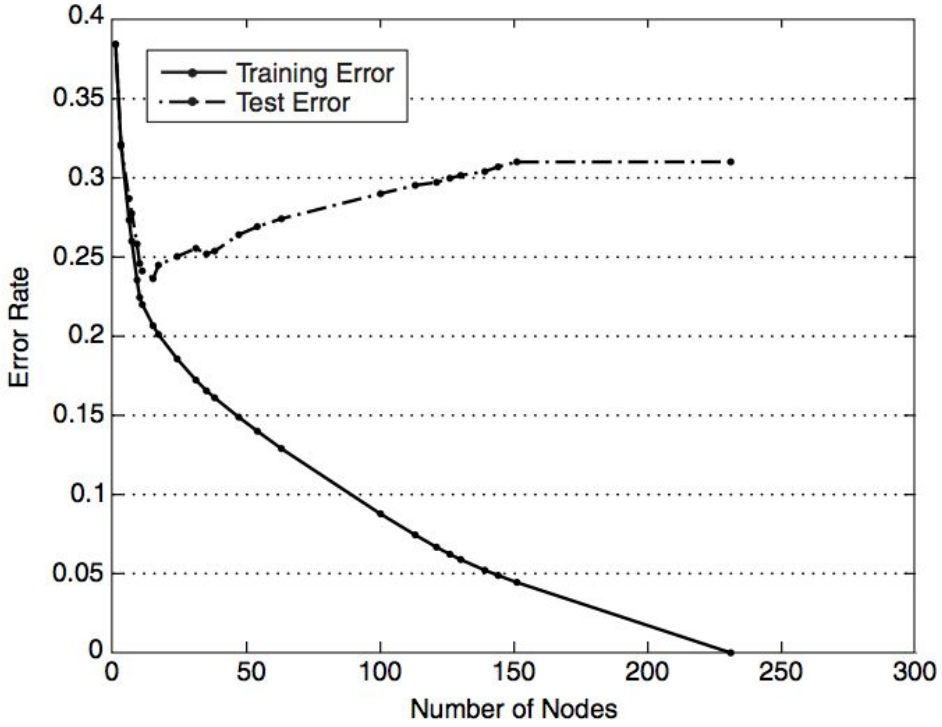


Image Source

Model Selection

Run cross validation to find a model that appropriately balances complexity and generalization capabilities: i.e., that optimizes the bias-variance tradeoff.

- High bias, low variance (underfitting)
  - Trees that aren't sensitive change from new data, but have a high error even in training
  - How to fix: Increase the complexity of the tree, e.g. Allow it to grow taller, increase the number of features
- Low bias, high variance (overfitting)
  - Trees that can capture more complex patterns during training but have trouble generalizing
  - How to fix: Set some minimum node size, only adding predictors whose children are big enough. Set some minimum improvement threshold, only adding predictors that are good enough

Example of Overfitting

What if a question reduces misclassification error to 0, but fails to generalize well?

E.g., Imagine a dataset of patients, a few of whom have a rare disease. If we split the data based on patient IDs, the misclassification error is 0. But when a new patient arrives with a unique ID, they are automatically classified as healthy, because the disease is rare!

Such a decision tree is utterly useless!

Key Design Decision

What size tree is best?

- Pre-pruning
  - Measure the marginal benefit of growing the tree, and stop when the impurity measure stops decreasing
  - Issue: most decision tree algorithms are greedy, while splitting may be subject to a horizon effect. That is, the benefits of a split may not immediate: the current split may have little effect on the impurity measure, but the ensuing split may be highly beneficial.
- Post-pruning alternative
  - Bundle sets of small leaf nodes into larger ones, assigning a decision by majority vote
  - Couple this idea with building a decision tree using held-out data to evaluate splits. Otherwise, the unique ID problem may preclude us from building any tree at all!

Quinlan’s Information Gain Ratio

Quinlan’s decision tree algorithms score questions based on an information gain ratio, rather than misclassification error.

Information gain aims to reduce Shannon entropy, just like our setup aimed to reduce misclassification error. Minimizing Shannon entropy is akin to minimizing impurities after splitting. But again, the rare disease example poses a problem, since it gains the most information, but in actual fact, we learn nothing.

Quinlan’s solution is to order questions based on a Gain Ratio: information gain divided by the average amount of information (no. of bits) after splitting, a quantity he calls SplitInfo.

One nice thing about Quinlan’s entropy-based approach is that it generalizes immediately beyond binary questions to decision trees with three- or four- or more-way questions.

Other issues with building decision trees

- How to fill in missing values
- How to split on numerical values
  - Temperature is in degrees rather than hot vs cold
  - One option is to bin values (temp > 75 vs temp < 75)
- How to handle noisy labels: identical observations with different labels